AICE2005 - Systematic Design

Formal Specification and Verification

Asieh Salehi Fathabadi
University of Southampton
2025-2026

Course Introduction

- Lecture 1: Introduction to Formal Methods
- Lecture 2: Formal Specification Techniques
- Lecture 3: Mathematical Foundations
- Lecture 4: Formal Verification Methods
- Lecture 5: Tools and Applications

LECTURE 4:

Formal Verification Methods

Lecture 4 Overview

- Verification Overview
- V & V : Validation and Verification
- Model Checking and Animation
- Theorem Proving and Proof Obligations (Event-B focus)

Learning Objectives:

- Understand model checking process and limitations
- Understand the role of animation in validation
- Apply proof obligation techniques in Event-B

Recap: Set theory and Logic Foundation

- **Basic Set Operations:**

  Union: $A \cup B$, Intersection: $A \cap B$ Difference: $A \setminus B$ Power Set: $\mathbb{P}(A)$

- **Relations and Functions:**

  Total Relation: $A \leftrightarrow B$, Partial Function: $A \nrightarrow B$, Injection: $A \rightarrowtail B$, Surjection: $A \twoheadrightarrow B$, Bijection: $A \rightarrowtail\!\!\!\!\twoheadrightarrow B$

- **Logic Foundation:**

  Negation: $\neg$, Conjunction: $P \wedge Q$, Disjunction: $P \vee Q$, Implication: $P \rightarrow Q$, Equivalence: $P \leftrightarrow Q$

  Universal: $\forall x$, Existential: $\exists x$

- **Linear Temporal Logic (LTL):**

  Next: $\circ\varphi$, Eventually: $\Diamond\varphi$, Always: $\Box$, Until: φ U

What is Formal Verification?

Formal verification is the use of **mathematical methods** to **prove or disprove** the **correctness** of a system with respect to its **specification**.

Verification vs Validation

- **Verification: "Are we building the System right?"**
  - Mathematical correctness
  - Formal proofs
- **Validation: "Are we building the right system?"**
  - Stakeholder requirements satisfaction
  - User acceptance, testing

Verification & Validation
V & V

Model Checking for Verification

Purpose: Automatically explore **ALL possible states** to verify safety properties

What is Model Checking?

Automatic exploration of your formal model where a tool checks ALL possible states to verify that safety properties always hold.

Model Checking for Event-B (ProB):

- **State Space Exploration:** Generate all reachable states
- **Invariant Checking:** Verify invariants hold in all states
- **Deadlock Detection:** Find states with no enabled events

State Explosion Problem:

- Consider ATM with multiple cards and accounts:
- 10 card types × 5 account types × 4 session states × 3 PIN attempt counts = 600 states
- Add concurrency: 2 ATMs = 600^2 = 360,000 states
- **Mitigation: Use abstraction, limit parameter ranges**

Model Checking Example

```
MACHINE DoorControl
SEES DoorContext
VARIABLES door train
INVARIANTS
        door ∈ DOOR_STATE
        train ∈ TRAIN_STATE
        door = OPEN ⇒ train = PRESENT
        train = MOVING ⇒ door = CLOSED
INITIALISATION
        door := CLOSED
        train := ABSENT
EVENTS
OpenDoors ≙ when
        train = PRESENT then
        door := OPEN end
CloseDoors ≙ when
         door = OPEN then
         door := CLOSE end
TrainArrives ≙ when
        train = ABSENT then
        train := PRESENT end
TrainDeparts ≙ when
        door = CLOSED then
        train := MOVING end
TrainContinues ≙ when
         train = MOVING then
        train := ABSENT end
END
```

Formal Specification:

SAFETY PROPERTY: **¬(door = OPEN ∧ train = MOVING)**

Model Checker explores ALL possible state sequences:

State 1: door=CLOSED, train=ABSENT ✓
↓ [TrainArrives]
State 2: door=CLOSED, train=PRESENT ✓
↓ [OpenDoors]
State 3: door=OPEN, train=PRESENT ✓
↓ [CloseDoors]
State 4: door=CLOSED, train=PRESENT ✓
↓ [TrainDeparts]
State 5: door=CLOSED, train=MOVING ✓
↓ [TrainContinues]
State 5: door=CLOSED, train=ABSENT ✓

Model Checker Result: ✓ Safety PROPERTY HOLDS

Model Checking Example

```
MACHINE DoorControl
SEES DoorContext
VARIABLES door train
INVARIANTS
        door ∈ DOOR_STATE
        train ∈ TRAIN_STATE
        door = OPEN ⇒ train = PRESENT
        train = MOVING ⇒ door = CLOSED
INITIALISATION
        door := CLOSED
        train := ABSENT
EVENTS
OpenDoors ≙ when
        train = PRESENT then
        door := OPEN end
TrainArrives ≙ when
        train = ABSENT then
        train := PRESENT end
TrainDeparts ≙ when
        door = CLOSED then
        train := MOVING end
TrainContinues ≙ when
         train = MOVING then
        train := ABSENT end
END
```

BUGGY MODEL (With Deadlock)

Bug: CloseDoors Event Missing

Deadlock Scenario:

State 1: door=CLOSED, train=ABSENT ✓

↓ [TrainArrives]

State 2: door=CLOSED, train=PRESENT ✓

↓ [OpenDoors]

State 3: door=OPEN, train=PRESENT ✓

⚠ **DEADLOCK DETECTED** ⚠

System is stuck! No events can execute!

Animation for Validation

Purpose: Simulate **specific scenarios** with domain experts to validate requirements

What is Animation?

Interactive simulation of your formal model where you execute events step-by-step to see how the system behaves.

Step	Current State	Expert Action	New State
1	door=CLOSED train=ABSENT	"Train arrives" → TrainArrives	door=CLOSED train=PRESENT
2	door=CLOSED train=PRESENT	"Open doors" → OpenDoors	door=OPEN train=PRESENT
3	door=OPEN train=PRESENT	"Close doors" → CloseDoors	door=CLOSED train=PRESENT
4	door=CLOSED train=PRESENT	"Train departs" → TrainDeparts	door=CLOSED train=MOVING

Expert: ✓ "Yes! This is exactly how our system operates!"

Proof Obligations in Event-B

What are Proof Obligations?
Proof obligations are **mathematical formulas that must be proven** to ensure the correctness of Event-B specification.

- **Well-Definedness (WD):** Ensure all expressions are well-defined
(e.g. Division by zero check)

- **Invariant Preservation (INV):** Prove invariants are maintained by all events

- **Guard Strengthening (GRD):** In refinement, concrete guards must imply abstract guards

- **Action Simulation (SIM):** Concrete actions must simulate abstract actions

Proof Obligations Example

Prove that OpenDoors event preserve invariant invx

```
INVARIANTS
        invx: door = OPEN ⇒ train = PRESENT
EVENT
OpenDoors ≙ when
        train = PRESENT then
        door := OPEN
end
```

What we know BEFORE the event:

- Invariant holds:
  door = OPEN ⇒ train = PRESENT
- Guard: train = PRESENT

What the event does:

- door := OPEN

What we have AFTER the event:

- door' = OPEN (just assigned)
- train' = PRESENT (unchanged)

Check invariant after event:

door' = OPEN ⇒ train' = PRESENT

↓ ↓

TRUE ⇒ TRUE

Result: TRUE ⇒ TRUE = TRUE ✓

WHY IT WORKS:

The guard "train = PRESENT" protects us! We can only open the door when the train is present, so the invariant is always satisfied.

✅ PROOF OBLIGATION DISCHARGED

Industrial Verification Example*

Paris Metro Line 14



https://www.railwaysignalling.eu/formal-methods-railway-signalling-systems

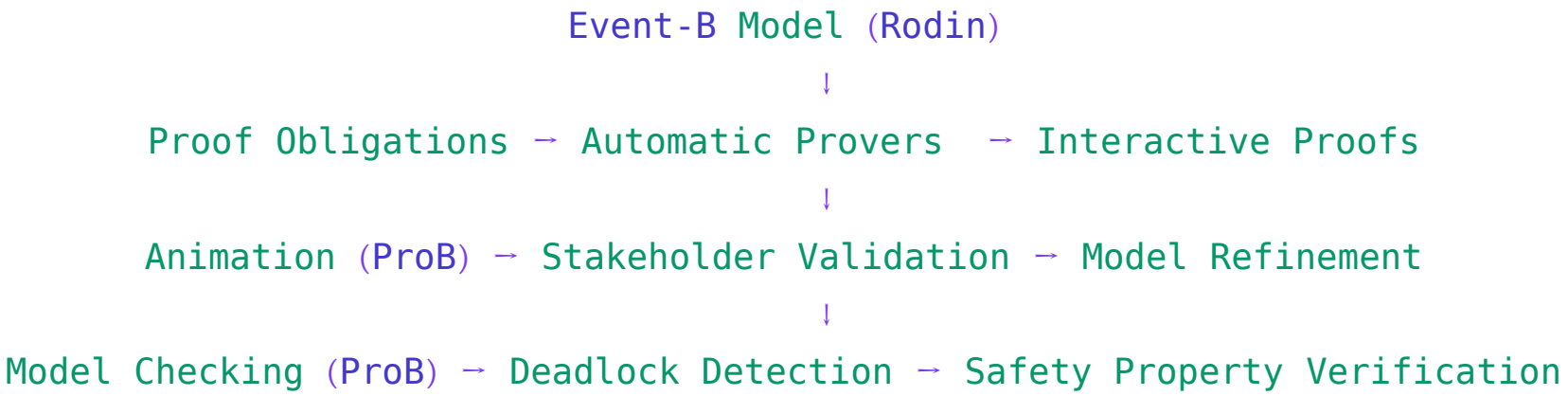
Verification Results:

- 27,800 proof obligations
- ~90% automatically discharged

Industrial Impact:

- Zero Safety Incidents since deployment in 1998

Best Practice



- **Start Abstract**: Begin with high-level model, refine gradually
- **Prove Incrementally**: Discharge proof obligations at each refinement step
- **Animate Early**: Validate with stakeholders before detailed refinement
- **Check Properties**: Use model checking for complex properties

Summery

- **Validation vs Veirificaiton**
  - Building the right system - Building the system Right
- **Model Checking for verification**
  - Explore all possible state to check safety properties
- **Animation for validation**
  - Validate specific scenario with doamin expert
- **Proof Obligations**
  - **Theorem proving in Event-B**

Let's read and explore

- Modeling in Event-B: System and Software Engineering
  by Jean-Raymond Abrial